TCP/UDP Echo client-server

Sample Input (client):
Enter message: Hello server

Output (client):
Connected to server.
Server replied: Hello server.

Output (server):
Server is listening
client connected
Received: Hello server
Echo sent successfully.

Exp 12.
Date:
14/9/26

End to End Communication at the Transport Layer.

a) Implement Echo client server using TCP/UDP sockets.

Aim:-

To implement an echo client-server application using TCP and UDP sockets and demonstrate end to end communication at the transport layer.

Algorithm:-

1. Choose a programming language such as python and import the socket module.
2. Create a server socket using TCP (SOCK_STREAM) or UDP (SOCK_DGRAM).
3. Assign an IP address and port number to the server.
4. For TCP, bind to socket, listen for connections and accept a client connection
5. For UDP, bind the socket and wait for incoming datagrams
6. Create a client socket and connect to the server.
7. Read a message entered by the client and transmit it to the server.
8. Make the server return for received message to the client

Sample Input (client)

Enter message: Hi

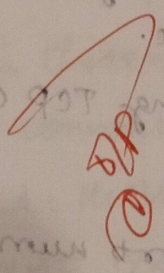
Sample output (server)

Client: Hi

Server: Hello!

Sample output (client):

Server: Hello!



9. Display the echoed message on the client terminal.

10. Close the sockets after communication is completed.

(b) Implement a chat program using socket programming.

Aim:-

To implement a client-server chat application using socket programming for exchanging message b/w computers.

Algorithm/Procedure:

1. Import the socket module in the selected programming language.
2. Create a server socket and bind it to a suitable IP address and port
3. Configure the server to listen for incoming client connections.
4. Create a client socket and connect it to the server.
5. Allow both the client and server to enter message.
6. Transmit the message using the socket se and receive operation
7. Display the received messages on the respective terminals.

8. Continue communication until either side enters an exit command.

9. Close the connections and release the socket resources.

Result:-

A chat application was implemented successfully using socket programming. The client and server exchanged messages through a network connections.